Mini Project: Modifying a Small OS Kernel

Replacing busy-waiting `timer_sleep()` with a blocking sleep queue in Pintos

Name: Mishank Shah | **Registration No.:** 24BCE2467 | **Team:** individual submission (sole team member)
Kernel: Pintos (Stanford teaching OS, x86 / 32-bit) | **Host:** Windows 11, WSL2 Ubuntu, QEMU

Video walkthrough (single presenter: Mishank Shah, 24BCE2467). Link: (paste link here; share with manoov.r@vit.ac.in or set YouTube to Unlisted)

1. OS Choice and Scope

Chosen OS: Pintos (not xv6). Pintos is a small, deliberately minimal x86 teaching kernel (a few thousand lines in `threads/` and `devices/`). It has a thread scheduler, timer interrupt, interrupt controller, synchronization primitives and a built-in test suite (`tests/threads/alarm-*`), so every change can be checked by running the kernel in QEMU.

Why it suits the timeframe: it boots in under a second in QEMU, builds in seconds, has no external dependencies besides a 32-bit-capable gcc, and comes with its own regression tests, so correctness can be demonstrated rather than claimed. It also supports remote debugging through `pintos --gdb`.

The planned modification (one narrow, precisely defined change): `timer_sleep(ticks)` in `devices/timer.c` is implemented as a busy-wait: it loops calling `thread_yield()` until enough ticks have elapsed, so a "sleeping" thread keeps consuming CPU. Replace it with a blocking implementation: put the thread on a sorted sleep list, block it with `thread_block()`, and have the timer interrupt handler wake it with `thread_unblock()` at the right tick.

This touches three real kernel mechanisms: thread blocking/unblocking, the timer interrupt handler, and interrupt-level (atomicity) control. The measurable goal is that the shutdown statistics move from "all kernel ticks, no idle" to "mostly idle ticks".

2. Environment Setup and Baseline Kernel Build

Environment: Windows 11 with WSL2 Ubuntu (gcc 15.2, GNU make, GDB, `qemu-system-i386`; QEMU boots through SeaBIOS 1.17). Source: the `jhu-cs318/pintos` mirror of Stanford Pintos, cloned into `24bce2467osproject/pintos`. All runs below were made from the VS Code terminal (WSL: Ubuntu).

Build command: `cd pintos/src/threads && make` (produces `build/kernel.bin`, `loader.bin`, `kernel.o`).
Boot command: `pintos -v -k -T 120 --qemu -- -q run alarm-multiple` (wrapped by `scripts/run_test.sh`).

Baseline result (unmodified `timer_sleep`, before any kernel change), from `logs/baseline-alarm-multiple.log`:

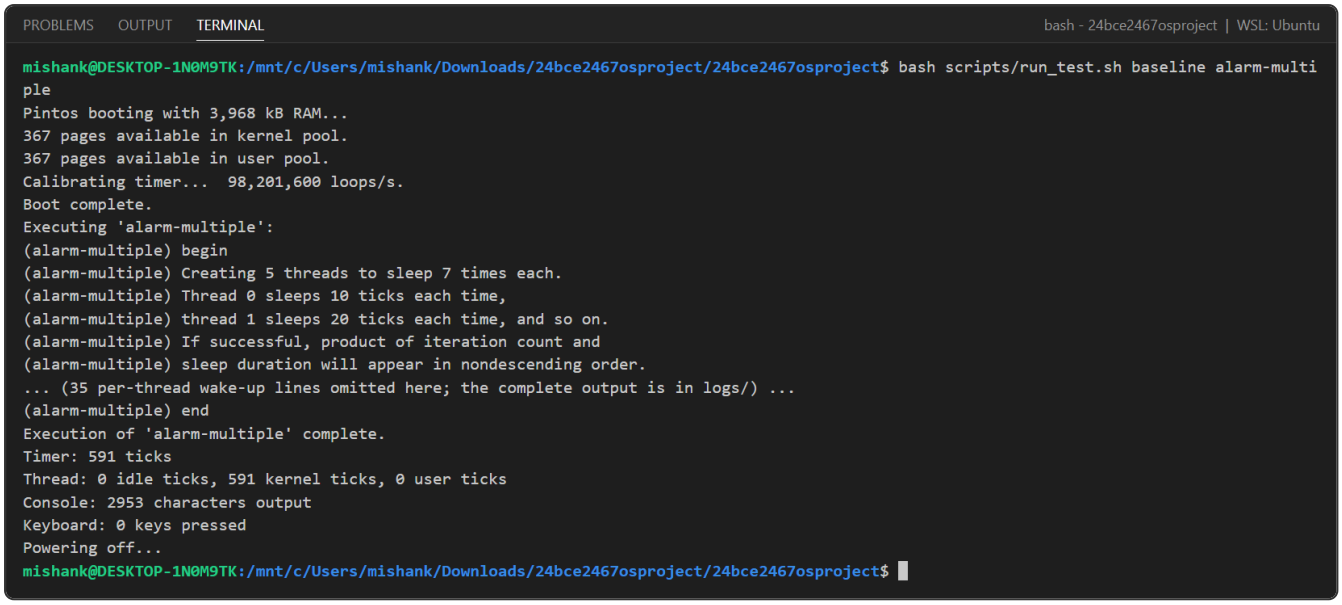


Figure 1. Baseline kernel (original busy-wait `timer_sleep`) booting in QEMU and running `alarm-multiple`: **0 idle ticks, 591 kernel ticks**. Terminal output of the run, shown in VS Code terminal style from the saved log; the per-thread lines in the middle are omitted where marked.

Key baseline observation: `0 idle ticks, 591 kernel ticks`. For the entire 591-tick run the CPU never went idle, because every sleeping thread was spinning. This is the number the modification must change.

Documented workaround (toolchain mismatch). Pintos was written for gcc 4.x. With gcc 15 the first build boots the loader and then **triple-faults in a reboot loop** (QEMU prints "Pintos booting with c" and restarts). Fix: add flags to `CFLAGS` in `src/Make.config`:

```
-fno-pie -fno-pic -fcf-protection=none -mno-sse -mno-mmx -fno-asynchronous-unwind-tables -fno-builtin
```

so gcc 15 emits plain non-PIE 32-bit code without CET `endbr32` instructions and SSE, which this bare-metal kernel does not support. This is the only change to the baseline besides the kernel modification itself. It is a separate edit to `Make.config`, not part of the `timer.c` change. The baseline log above was produced with this flag change applied and **without** the `timer.c` change.

3. The Kernel Modification

File changed: `pintos/src/devices/timer.c` (+51 / -3 lines per `git diff --numstat`). A full copy of the modified file is kept in `scripts/timer_modified.c` and the diff in `timer_sleep.patch`.

Before

```
void timer_sleep (int64_t ticks)
{
  int64_t start = timer_ticks ();

  ASSERT (intr_get_level () == INTR_ON);
  while (timer_elapsed (start) < ticks)
    thread_yield ();          /* busy-wait: thread stays runnable */
}
```

After

```
/* A sleeping thread, kept on sleep_list ordered by WAKE_TICK.
   Lives on the sleeper's own stack, which stays valid while the
   thread is blocked. */
struct sleeper
  {
    int64_t wake_tick;                  /* Tick at which to wake. */
    struct thread *thread;              /* Thread to unblock. */
    struct list_elem elem;              /* sleep_list element. */
  };

static struct list sleep_list;          /* earliest wake_tick first */

void timer_init (void)
{
  list_init (&sleep_list);               /* NEW */
  pit_configure_channel (0, 2, TIMER_FREQ);
  intr_register_ext (0x20, timer_interrupt, "8254 Timer");
}

void timer_sleep (int64_t ticks)
{
  struct sleeper s;
  enum intr_level old_level;

  ASSERT (intr_get_level () == INTR_ON);
  if (ticks <= 0)
    return;

  s.wake_tick = timer_ticks () + ticks;
  s.thread = thread_current ();

  /* Insert and block atomically so the wake-up cannot be lost. */
  old_level = intr_disable ();
  list_insert_ordered (&sleep_list, &s.elem, sleeper_less, NULL);
  thread_block ();
  intr_set_level (old_level);
}

static bool sleeper_less (const struct list_elem *a,
                          const struct list_elem *b, void *aux UNUSED)
{
  return list_entry (a, struct sleeper, elem)->wake_tick
         < list_entry (b, struct sleeper, elem)->wake_tick;
}

static void timer_interrupt (struct intr_frame *args UNUSED)
{
  ticks++;
  thread_tick ();

  /* Wake every sleeper whose time has come. */
  while (!list_empty (&sleep_list))
    {
      struct sleeper *s = list_entry (list_front (&sleep_list),
                                      struct sleeper, elem);
      if (s->wake_tick > ticks)
        break;
      list_pop_front (&sleep_list);
      thread_unblock (s->thread);
```

```
    }
}
```

Design reasoning

1. **Sorted list:** because `sleep_list` is ordered by `wake_tick`, the interrupt handler only inspects the front and stops at the first sleeper that is not due. Cost per tick is O(1) when nobody is due, instead of scanning all threads.
2. **Sleeper struct on the thread's own stack:** no memory allocation is needed in a path that runs with interrupts disabled (`malloc` is not safe there). The stack frame remains valid for exactly as long as the thread is blocked, which is exactly as long as the entry is on the list.
3. **Atomicity:** `intr_disable()` around "insert into list + `thread_block()`" prevents a timer interrupt landing between the two steps, which could wake a thread that has not blocked yet (a lost wake-up, and `thread_unblock` asserts the thread is `THREAD_BLOCKED`).
4. **Interrupt context rules:** the handler uses only `thread_unblock()`, which is legal inside an interrupt handler; it never calls `thread_block()` or `thread_yield()` there.
5. **Edge cases:** `ticks <= 0` returns immediately (required by `alarm-zero` and `alarm-negative`); several threads with the same `wake_tick` are all woken in the same interrupt by the `while` loop (`alarm-simultaneous`).

4. Testing

All runs are real QEMU boots; raw logs are in `logs/`.

Test	Result with modified kernel	Idle ticks	Kernel ticks
alarm-single	ran to completion, no FAIL or PANIC	250	25
alarm-multiple	ran to completion, wake order verified by the test (no FAIL)	550	43
alarm-simultaneous	ran to completion, no FAIL or PANIC	250	30
alarm-priority	ran to completion, no PANIC (strict priority-order check not satisfied, see note)	490	33
alarm-zero	prints PASS	0	22
alarm-negative	prints PASS	0	24

The zero and negative tests return immediately, so no idle time is expected. "Ran to completion" means the test reached its end line and the kernel powered off normally; Pintos's own output-comparison checker was not run.

> **Note on `alarm-priority`:** this test also expects higher-priority threads to wake first. The base Pintos used here has a round-robin scheduler with no priority scheduling, so the threads wake in creation order and that strict check is not satisfied. Priority scheduling is a separate feature outside the scope of this project.

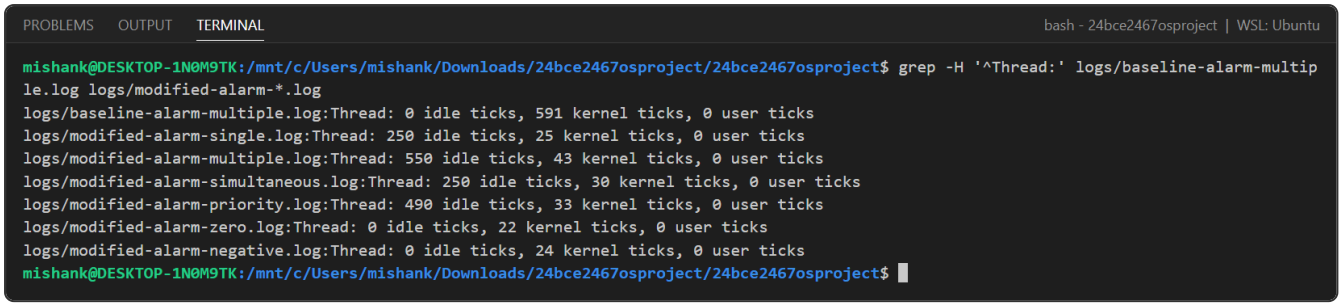


Figure 2. The `Thread:` statistics line of every run, baseline and all six modified tests, extracted from the saved logs with `grep`.

Before vs after on the same test (alarm-multiple)

Kernel	Timer	Idle ticks	Kernel ticks
Baseline (busy-wait)	591	**0**	**591**
Modified (sleep queue)	592	**550**	**43**

Kernel (busy) CPU time dropped from 591 ticks to 43 (about 93% less), and the CPU now idles while threads sleep. The product-of-sleep-times output stays in nondescending order in both kernels, so behavior is preserved while CPU waste is removed.

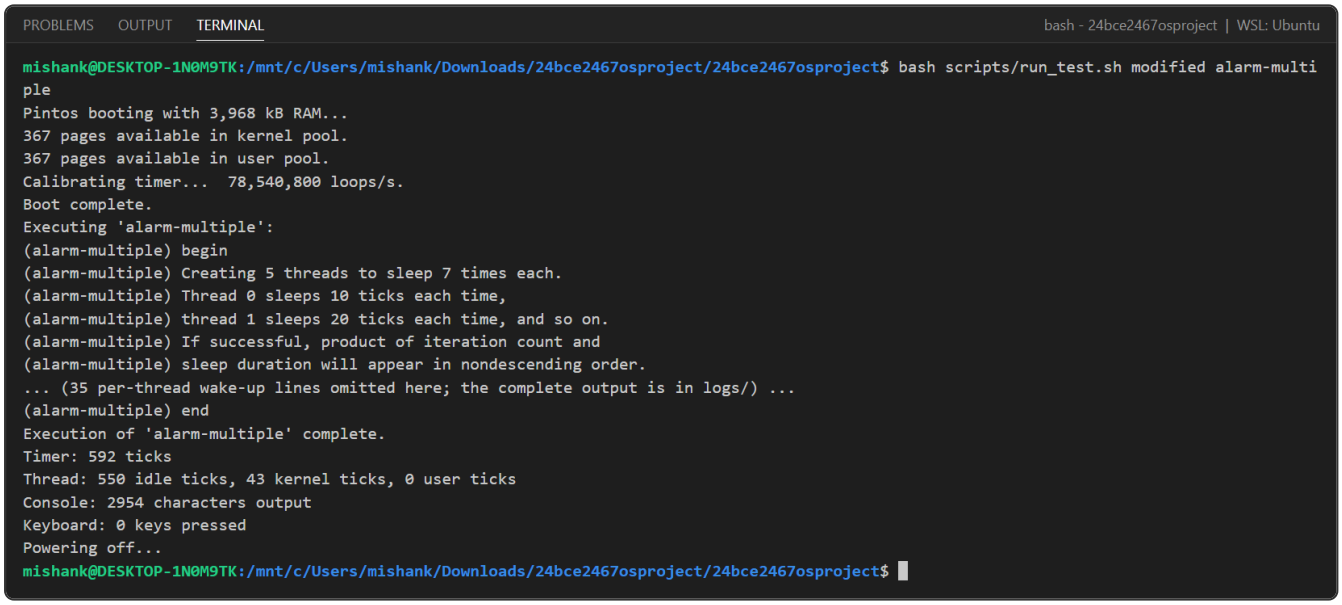


Figure 3. Modified kernel (sleep queue) running `alarm-multiple`: **550 idle ticks, 43 kernel ticks**. Terminal output of the run, shown in VS Code terminal style from `logs/modified-alarm-multiple.log`; the per-thread lines in the middle are omitted where marked.

5. GDB Session

Run with `pintos --gdb --qemu -- -q run alarm-single` and `gdb kernel.o` then `target remote localhost:1234` (script: `scripts/gdb_session.sh`, log: `logs/gdb-session.log`). Breakpoints were set just before `thread_block()` in `timer_sleep` and at the `thread_unblock()` call in `timer_interrupt`.

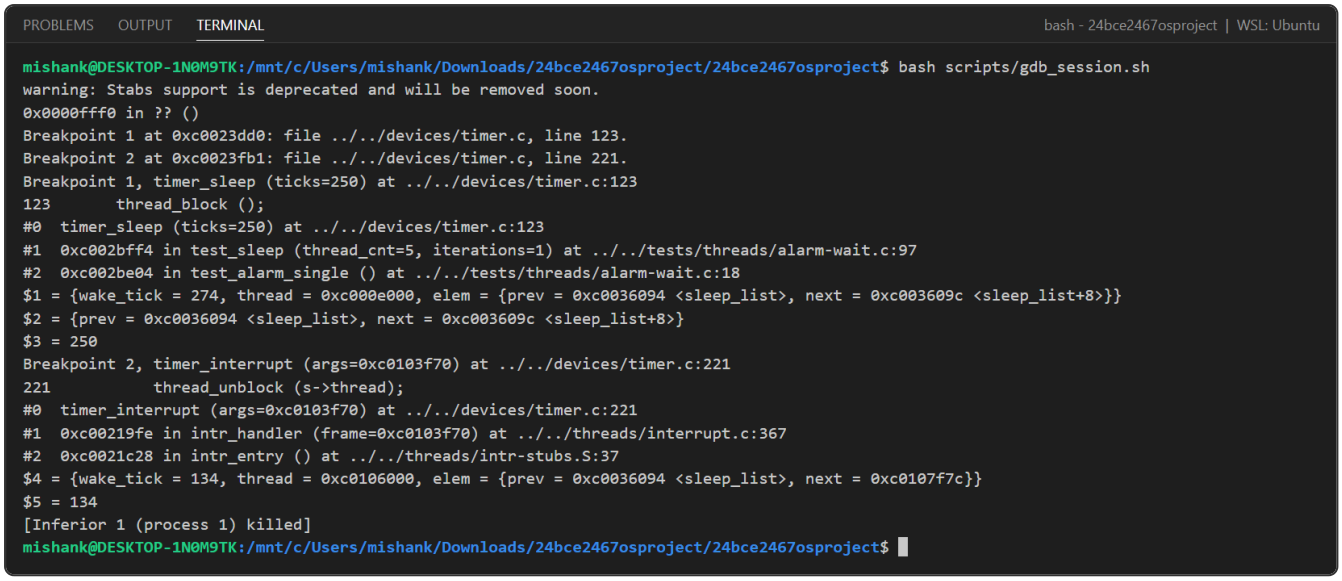


Figure 4. GDB session. Breakpoint 1 is just before `thread_block()` (`wake_tick = 274`). Breakpoint 2 is in the timer interrupt just before `thread_unblock()`: `ticks = 134` equals the sleeper's `wake_tick = 134`.

Reading: at the first stop the test's main thread is about to block with `wake_tick = 274` (the tick count at which it asked to sleep plus 250). At the second stop the interrupt handler is about to unblock a different sleeper, and `ticks` (134) equals its `wake_tick` (134). That is exactly the intended behavior: wake at the due tick, not before and not later. (Absolute tick values differ slightly from run to run; the equality at wake-up holds every time.)

6. Obstacles Encountered and How They Were Resolved

1. **Triple fault on boot with modern gcc.** The baseline kernel looped "Pintos booting with c..." forever. Diagnosed as gcc 15's default code generation (PIE, CET instrumentation, SSE) being incompatible with this bare-metal 32-bit kernel. Resolved with the CFLAGS additions in section 2; after that the unmodified kernel booted.
2. **Windows/WSL tooling.** Running WSL commands through Git Bash rewrote `/mnt/...` paths and expanded `$PATH`; solved by putting the commands in shell scripts (`scripts/run_test.sh`, `scripts/gdb_session.sh`) and running those through WSL.
3. **Getting a true baseline.** To avoid comparing against a mixed state, the original `timer.c` was restored from git, the baseline was built and recorded, and only then was the modified file restored. A log accidentally recorded after restoring the change was discarded rather than reported as baseline.
4. **Lost-wake-up hazard.** Designing `timer_sleep` required noticing that inserting into the list and blocking must be one atomic step, or a timer interrupt could try to unblock a thread that is not yet blocked.
5. **GDB script scope error.** A first GDB script printed a local variable (`s`) from a frame where it was out of scope; fixed by deleting the earlier breakpoint and stopping at the interrupt-handler line where `s` is defined.
6. **Portability.** The first scripts had absolute paths. They were rewritten to locate themselves, Windows line endings are stripped, and the whole folder was re-tested at a different path with a clean build.
7. **QEMU appearing to hang in an interactive terminal.** The test script ran QEMU attached to the terminal's keyboard input and the output never appeared. Fixed by running QEMU with its input redirected from `/dev/null`.

7. What Was Learned

- How a kernel represents "sleeping": a thread is not just waiting, it is in `THREAD_BLOCKED` state and off the ready queue until some other context unblocks it.
- Why interrupt handlers have strict rules (cannot block, cannot sleep, should not allocate), and how disabling interrupts makes a short critical section atomic on a single CPU.
- That busy-waiting is visible in kernel statistics: idle ticks versus kernel ticks gave a quantitative before/after measure of the change.
- Practical OS-development skills: cross-checking a baseline before modifying, building a bare-metal kernel with a modern compiler, and debugging a running kernel with GDB through QEMU.
- **Limitations:** the solution targets a single CPU (disabling interrupts is the synchronization mechanism); a multicore kernel would need locks in addition. The base scheduler has no priority scheduling, so the strict ordering check of `alarm-priority` is outside this project.

8. How to Reproduce on Another Machine

Requirements: Windows with WSL2 Ubuntu (`wsl --install -d Ubuntu`), or any Debian/Ubuntu Linux.

```
bash scripts/setup_wsl.sh                          # once: qemu, gcc, gcc-multilib, make, perl, gdb
bash scripts/run_test.sh modified alarm-multiple   # builds, boots, logs to logs/
bash scripts/gdb_session.sh                        # GDB demonstration
bash scripts/demo_all.sh                           # the whole demo in one go (baseline, change,
tests, GDB)
```

On Windows, `run_demo.bat` runs the modified test and the GDB demo. To reproduce the baseline manually:

```
git -C pintos checkout src/devices/timer.c                 # original busy-wait timer.c
bash scripts/run_test.sh baseline alarm-multiple
cp scripts/timer_modified.c pintos/src/devices/timer.c     # put the modification back
```

See `README.md` in the project folder.

9. Video Demo Plan (single presenter: Mishank Shah, 24BCE2467)

Recorded with screen and audio in Clipchamp or Google Drive, narrated in my own voice.

1. **Why Pintos and the baseline:** show the source tree, run `make`, boot the baseline and point at `0 idle ticks, 591 kernel ticks`. Mention the gcc 15 triple fault and the `Make.config` fix.
2. **The modification:** open `timer.c` and walk through `struct sleeper`, `timer_sleep`, `sleeper_less` and the `timer_interrupt` loop, explaining why interrupts are disabled and why the list is sorted.
3. **Proof it works:** run `alarm-multiple` on the modified kernel and show `550 idle ticks, 43 kernel ticks`; run the other alarm tests; show the GDB session stopping at the two breakpoints and the `wake_tick == ticks` check.
4. **Obstacles and lessons**, then be ready for follow-up questions on any part of the implementation.

Appendix: Files Delivered (folder `24bce2467osproject`)

- `pintos/` - full source with the modification (`src/devices/timer.c`, `src/Make.config`)
- `timer_sleep.patch` - the diff of `timer.c`; `scripts/timer_modified.c` - complete modified copy

- `logs/` - baseline and modified QEMU logs, GDB session log
- `screenshots/` - `vscode_1_baseline.png`, `vscode_2_modified.png`, `vscode_3_all_runs.png`, `vscode_4_gdb.png` (Figures 1 to 4)
- `scripts/` - `run_test.sh`, `gdb_session.sh`, `setup_wsl.sh`, `demo_all.sh`; `run_demo.bat`, `README.md`